

Solution to Problem 3 in Problem Sheet - Week 4

MATH70102 Unsupervised Learning

Dr Mikko Pakkanen, Department of Mathematics, Imperial College London

```
In [1]: import matplotlib.pyplot as plt
import numpy as np
from scipy.integrate import quad
from sklearn.manifold import MDS
from sklearn.metrics import pairwise_distances

plt.style.use("ggplot")
```

Part (a)

Define samples $\mathbf{x}_1 = (-1, 10)$, $\mathbf{x}_2 = (0, 0)$ and $\mathbf{x}_3 = (1, 10)$:

```
In [2]: x1 = np.array([-1.0, 10.0])
x2 = np.array([0.0, 0.0])
x3 = np.array([1.0, 10.0])
```

Compute their Euclidean distance matrix:

```
In [3]: X = np.vstack([x1, x2, x3])
d_euclid = pairwise_distances(X, metric="euclidean")
d_euclid
```

```
Out[3]: array([[ 0.          , 10.04987562,  2.          ],
               [10.04987562,  0.          , 10.04987562],
               [ 2.          , 10.04987562,  0.          ]])
```

Part (b)

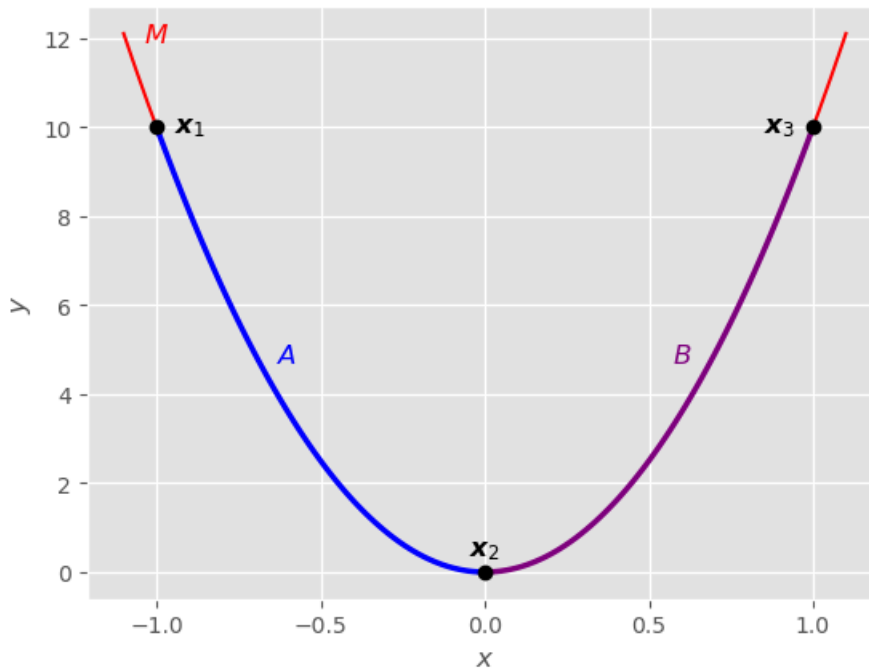
Let us first plot $\mathbf{x}_1 = (-1, 10)$, $\mathbf{x}_2 = (0, 0)$ and $\mathbf{x}_3 = (1, 10)$ and the manifold $M = \{(x, y) \in \mathbb{R}^2 : y = 10x^2\}$:

```
In [4]: x_M = np.linspace(-1.1, 1.1, 301)
x_A = np.linspace(-1.0, 0.0, 101)
x_B = np.linspace(0.0, 1.0, 101)

plt.plot(x_M, 10 * x_M**2, color="red")
plt.plot(x_A, 10 * x_A**2, lw=2.5, color="blue")
plt.plot(x_B, 10 * x_B**2, lw=2.5, color="purple")
plt.plot(x1[0], x1[1], "o", color="black")
plt.plot(x2[0], x2[1], "o", color="black")
plt.plot(x3[0], x3[1], "o", color="black")

plt.text(
    x1[0] + 0.1, x1[1], r"\boldsymbol{x}_1", fontsize=12, ha="center", va="center"
)
plt.text(
    x2[0], x2[1] + 0.5, r"\boldsymbol{x}_2", fontsize=12, ha="center", va="center"
)
plt.text(
    x3[0] - 0.1, x3[1], r"\boldsymbol{x}_3", fontsize=12, ha="center", va="center"
)
plt.text(
    -1.1 + 0.1, 10 * 1.1**2, r"$M$", color="red", fontsize=12, ha="center", va="center"
)
plt.text(
    -0.7 + 0.1, 10 * 0.7**2, r"$A$", color="blue", fontsize=12, ha="center", va="center"
)
plt.text(
    0.7 - 0.1, 10 * 0.7**2, r"$B$", color="purple", fontsize=12, ha="center", va="center"
)
```

```
plt.xlabel(r"$x$")
plt.ylabel(r"$y$")
plt.show()
```



The geodesic distance between x_1 and x_2 is the length of the segment A:

```
In [5]: def le(x):
        return np.sqrt(1 + (20 * x) ** 2)

A, error = quad(le, -1.0, 0.0)
print("Length of A:", A)
print("Error:", error)
```

Length of A: 10.10472979397512
Error: 6.13577470101484e-09

The geodesic distance between x_2 and x_3 is the length of the segment B which, by symmetry, equals that of A:

```
In [6]: B = A
        print(B)
```

10.10472979397512

And the geodesic distance between x_1 and x_3 is the sum of the two:

```
In [7]: print(A + B)
```

20.20945958795024

Finally, let us create a distance matrix:

```
In [8]: d_geo = np.array([[0, A, A + B], [A, 0, B], [A + B, B, 0]])
        d_geo
```

```
Out[8]: array([[ 0., 10.10472979, 20.20945959],
               [10.10472979,  0., 10.10472979],
               [20.20945959, 10.10472979,  0.]])
```

Part (c)

Perform MDS on the Euclidean distances using the class `MDS` from the module `sklearn.manifold`:

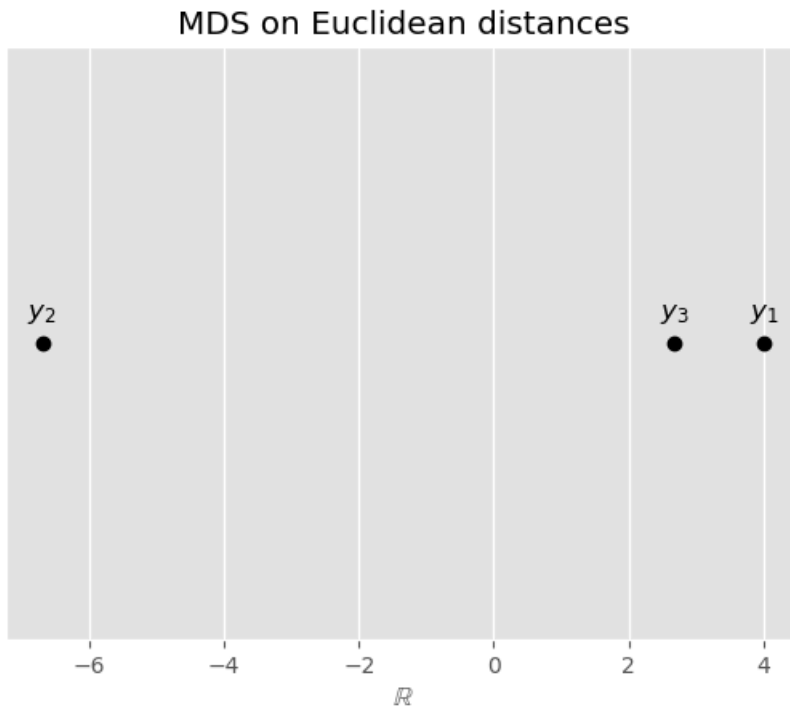
```
In [9]: mds_euclid = MDS(
        n_components=1,
        dissimilarity="precomputed",
        random_state=1234,
        normalized_stress=False,
```

```

)
y_euclid = mds_euclid.fit_transform(d_euclid)

plt.plot(y_euclid, [0.0, 0.0, 0.0], "o", color="black")
plt.text(y_euclid[0], 0.01, r"$y_1$", fontsize=12, ha="center", va="center")
plt.text(y_euclid[1], 0.01, r"$y_2$", fontsize=12, ha="center", va="center")
plt.text(y_euclid[2], 0.01, r"$y_3$", fontsize=12, ha="center", va="center")
plt.xlabel(r"$\mathbb{R}$")
plt.gca().axes.get_yaxis().set_visible(False)
plt.ylim(-0.1, 0.1)
plt.title("MDS on Euclidean distances")
plt.show()

```



Part (d)

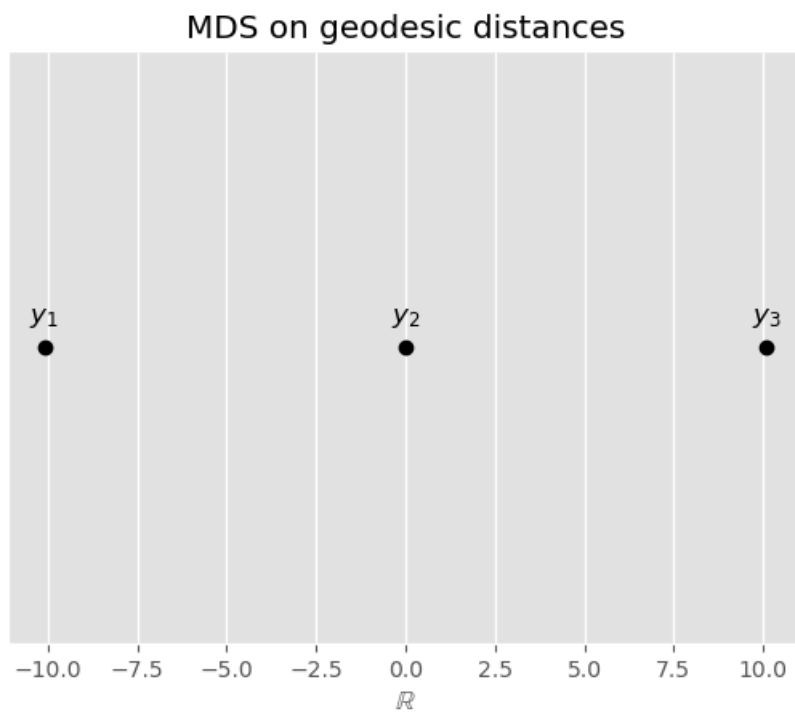
Apply MDS to the geodesic distances:

```

In [10]: mds_geo = MDS(
            n_components=1,
            dissimilarity="precomputed",
            random_state=1234,
            normalized_stress=False,
        )
y_geo = mds_geo.fit_transform(d_geo)

plt.plot(y_geo, [0.0, 0.0, 0.0], "o", color="black")
plt.text(y_geo[0], 0.01, r"$y_1$", fontsize=12, ha="center", va="center")
plt.text(y_geo[1], 0.01, r"$y_2$", fontsize=12, ha="center", va="center")
plt.text(y_geo[2], 0.01, r"$y_3$", fontsize=12, ha="center", va="center")
plt.xlabel(r"$\mathbb{R}$")
plt.gca().axes.get_yaxis().set_visible(False)
plt.ylim(-0.1, 0.1)
plt.title("MDS on geodesic distances")
plt.show()

```



This result is faithful to the geometry of the manifold M unlike the result in (c).